GC215: Command-tag Generalization and Conditional Decoder Sharing

A new binary 215-state candidate for the consistency of ZFC

Prepared for Vladimir Reshetnikov's ProveIt project
AI-assisted construction with separate logical, source, and machine reviews

21 October 2026
Repository pin: `34b63b3b197b0a5f285a229d91bc163fcc42ab93`

Abstract

We construct GC215, a deterministic one-tape, two-symbol Turing machine with 215 nonhalting states, including its blank-tape initialization. Using the accepted ProveIt numerical-calculus calibration at the pinned revision, an ordinary mathematical argument formalizable in ZFC proves

$$\neg\,\mathrm{Halt}(\mathrm{GC215}, \mathrm{blank}) \iff \mathrm{Con}(\mathrm{ZFC}).$$

The machine changes the proof language: an axiom command generalizes its cached theorem using the command's private parameter rather than its incoming arithmetic counter. This removes a counter-loading command from compiled generalization steps. The extra source decoder role is paid for by a new conditional merge using an existing tape marker and existing helper states. The exact table passes all 6,151 arbitrary-natural primitive contracts, covering all 430 rows, and its complete blank computation reaches the calibrated source frame after 68,710 transitions. The packet also gives a new 107-node proof that the sixth numerical axiom may be restricted to matrices larger than their binder while preserving every raw theorem code. No 214-state machine was obtained. The new table has no new compiled Lean/Rocq endpoint in this packet; its proof uses explicit inherited calibration together with fresh ordinary source, runtime, and startup proofs.

Contents

1 Result, exact convention, and what is new **3**
1.1 The proof boundary . . . 3

2 A two-operand source change and a reproducible allocation **3**

3 The conditional fold that pays for the new source node **4**

4 Complete calibration of the private-binder transfer source **5**
4.1 The retained numerical calculus . . . 5
4.2 Exact commands and field order . . . 6
4.3 Structural macros and a strong proof compiler . . . 7
4.4 Exact packing and cofinal successful candidates . . . 8
4.5 An all-natural protected-tail theorem . . . 8
4.6 Application to the exact measured blank frame . . . 10

5 Physical verification and its exact scope **11**
5.1 Universal primitive contracts . . . 11
5.2 A complete blank-prefix witness . . . 11
5.3 Comparison with current inputs . . . 11

6 Formalism-side work: an axiom restriction that survives exact codes **12**
6.1 A quantified axiom sampler with a strict numerical bound . . . 12

7 Other calibrated encodings and command interfaces **13**
7.1 Sixteen calibrated formula encodings from one native calculus . . . 13
7.2 Moving the sole falsity guard to a register that is about to be overwritten . . . 14

8 What the unsuccessful reductions establish **16**
8.1 Strict fourth-parameter dominance is only partially recovered . . . 16
8.2 Why deleting the last scanner state did not work . . . 17
8.3 Local machine synthesis has a bounded negative result . . . 17
8.4 A rejected transfer did not receive a calibration by analogy . . . 17

9 Reproduction, integration, and trust **17**

Attribution **18**

A Complete dense transition table **19**

1 Result, exact convention, and what is new

GC215 has one two-way infinite tape, initially all zero; a head initially at position zero; symbols $0, 1$; and moves L, R only. Every working state has one transition for each read symbol. The dense presentation has working states $0, \ldots, 214$, initial state 0, and a separate halt 215 with no outgoing rows. A native presentation retains historical diagnostic labels; its halt is 254 and its dispatcher is 84. The dense dispatcher is 81. The complete dense table is printed in Appendix A.

Measured or checked quantity	GC215
Working states / transition rows	215 / 430
Source primitive occurrences / distinct subtrees	4,732 / 202
Maximum source address length / zero run	44 / 6
Universal guarded primitive cases	6,151
Symbolic proof rules in the fresh runtime check	1,617,679
Rows covered by those cases	all 430
Minimum constant term of a duration	5
Complete blank initialization prefix	68,710 steps
Checkpoint one-cells / zero-valued spare registers	100 / 63

The exact delivered identities are:

Dense table, `machine/gc215.tm`
`ccabf998e346356edb33a829a8c2ab7a24b69f032349e9228f53e2295337ec0f`
Native table, `machine/gc215.native.tm`
`a5d8dfeb92a6ae843692c65b5eaf69b845b08b48d65dc011ba8a457e4faec67d`
Source, `source/gc215.sites.json`
`3b4e8203227f4be6d868ffdfb8d1a6311f5ac9a647dadf8386eb8f0de509ce57`

The immediate source and physical predecessor is the pending `ST215_Direct_Transfer_2026-10-21.zip` packet in the pinned repository [2]. That predecessor already has the direct single-cache-guard transfer command and a complete 215-state construction. Neither that transfer idea nor its original compiler is claimed as new here.

The present changes are substantive at both command and machine levels. The old AX command emits $Q_U(\text{old } h)$, where U is the incoming counter. GC215 emits $Q_C(\text{old } h)$, where C is supplied by the command tag. Its strong proof compiler therefore omits $J(i)$ in a generalization of binder i. For $i > 0$, this removes one MP command and its two numerical fields from that compiled proof block. The source has one more distinct subtree than ST215. A new physical merge of roles 129 and 310 returns the working-state count to 215. This is not a state renaming, a reflection, or just a different list of table rows.

No uniform total-runtime improvement is asserted. Numerical parameter sizes and proof words change, so a shorter block need not give a simple uniform binary-time comparison. The measured blank prefix is 1,796 steps shorter than ST215's 70,506-step prefix; this is only a startup comparison.

1.1 The proof boundary

The repository's primary K215 now has compiled Lean and Rocq endpoints [1]. Those endpoints concern that earlier exact table. We do not transfer a proof-assistant endpoint merely by preserving the state count. The present theorem is an ordinary ZFC-formalizable proof composed from: the inherited native-calculus/ZFC calibration, the fresh command compiler, the literal source change, the arbitrary-natural binary simulation, and the complete finite blank prefix. The inherited dominated-ρ_8 certificate is checked afresh but is only one part of that dependency chain.

All proof claims concern finite syntactic refutations. The construction does not decide whether ZFC is consistent, prove unconditional nonhalting, or establish a lower bound on the number of states required.

2 A two-operand source change and a reproducible allocation

The ordered primitive source uses increment, guarded decrement, break, continue, and halt. Registers are numbered $0, \ldots, 8$ and interpreted as $(N, T, W, A, D, H, X, Y, L)$ in the inherited convention. Register 3 is

the decoded residual command tag, not the logical parameter called A in an AX equation. Register 4 is the additive counter and register 5 is the cache. Confusing these two uses of "A" would change the proposed sampler.

Relative to ST215, the literal source modifications are exactly:

Ordinal	Source address	Old	New
44	`0011001000100`	`CD4`	`CD3`
49	`00110010001110`	`CI4`	`CI3`

All 4,732 source addresses and all guarded control-flow targets remain unchanged. Two independently organized CFG resolvers agree. The changed seven-leaf interval $[44, 51)$ is a closed copying macro. Starting with $T = X = 0$ and $C = c$, its first loop decrements C while incrementing T and X, and its second loop moves X back to C. It terminates with $(C, T, X) = (c, c, 0)$ and leaves every other register unchanged. The two loop variants are C and X respectively. This is an all-natural argument, also obtainable by conjugating the old copy macro by the interchange of registers 3 and 4.

The surrounding stack push is unchanged. Therefore the first generalized AX formula uses C in place of U, while all parameter registers and all subsequent AX computations retain their values. The check binds this closed macro to the exact two source sites; the arithmetic argument supplies the quantified semantic effect.

An allocator first produces a corrected 216-state table, delivered as `reference/gc216.native.tm`. It is then reduced by the local proof in the next section. Replay does not require that allocator or a solver: `source/table_recipe.json` lists all changes to the pinned ST215 table and the final merge. The verifier rebuilds the parent and candidate bytes exactly, then checks the dense state bijection.

The first allocation attempt accidentally overwrote an engine use of 134/0. It passed a decoder-prefix audit but failed the very first universal primitive case. That rejected table is not the candidate. The corrected allocation protects the row, and both corrected 216 and final 215 tables were fully checked. This is a concrete reason to distinguish source-node allocation from a binary runtime proof.

3 The conditional fold that pays for the new source node

The recompiled source using the C parameter in the generalization row has 202 distinct source subtrees. A corrected literal allocation first produces 216 working states. The required return to 215 states uses a new conditional decoder fold; no existing conditional-marker helper is copied into a spare state. An already present decoder chain supplies the required dispatch.

Let T denote the corrected 216-state table and N the resulting 215-state table. Define $\phi(310) = 129$ and $\phi(q) = q$ otherwise. The construction deletes the two rows of state 310, redirects every target 310 to 129, and replaces the remaining row 129/1 by $1L216$. All other rows are retained, subject only to the target redirection. In this table the sole redirected incoming row is 312/0.

Original state	Read 0	Read 1
129	$0R154$	$1R189$
310	$0R154$	$1R86$

The complete incoming-row census is particularly simple. State 129 has only the incoming row $211/1 = 1R129$, and state 310 has only the incoming row $312/0 = 0R310$. Neither state is the initial control. Therefore, on every execution starting in control 0, an arrival at old state 129 has a one immediately to its left, while an arrival at old state 310 has a zero there. This statement holds on arbitrary initial binary tapes.

Both old roles agree on read 0. On read 1, the new row moves left to state 216 and reads this origin marker. For marker one, its execution is

$$129 \xrightarrow{1L} 216 \xrightarrow{1R} 178 \xrightarrow{1R} 189.$$

For marker zero, the execution is

$$129 \xrightarrow{1L} 216 \xrightarrow{0R} 157 \xrightarrow{1L} 148 \xrightarrow{0R} 186 \xrightarrow{1R} 86.$$

Every write reproduces the symbol already present. Both paths end one cell to the right of the original head, in exactly the control required by the corresponding old role. No exterior tape cell is read. Thus one old step is simulated by respectively three or five new steps on read 1, and by one new step on read 0.

At every other old state, one step commutes with ϕ. Induction therefore matches the complete old computation from control 0 with the new computation at a sequence of boundaries having identical tapes and heads and controls related by ϕ. Each matching block has positive length at most five. Consequently T and N halt together, on every initial tape from control 0. The claim is not equivalence from arbitrary old control 310 with an arbitrary unconstrained left marker.

The independent checker reads the literal tables, verifies the complete incoming census and all four local blocks, checks all redirected rows, and checks that all 215 surviving working states remain graph reachable. It also produces a dense table with working controls $0, \ldots, 214$ and separate halt control 215, and verifies the exact state-label bijection. The native table uses separate halt control 254. Universal primitive simulation and the blank bootstrap are independently checked obligations; the finite fold proof alone does not establish their premises.

One allocator repair is essential to this construction. Row $134/0 = 0R17$ must be protected because the existing engine can enter it through $11/0 = 0L26$ and $26/0 = 0R134$. A prefix-only decoder audit does not detect a change to that engine use. The uncorrected 216-state allocation failed its first arbitrary-natural primitive contract, and is not a candidate.

4 Complete calibration of the private-binder transfer source

The source `source/gc215.sites.json` uses the original single cache guard. This section proves its complete command-level calibration. Its only logical-interface change is that the ninth AX output generalizes with the independently supplied private parameter C, rather than the incoming counter U. The argument below supplies both directions of the refutation equivalence, the exact wire packing, and the protected-tail bridge needed for the measured blank initialization. The source and binary runtime contracts are separate, explicitly identified premises; finite execution tests are not used to infer these unbounded mathematical statements.

4.1 The retained numerical calculus

For all natural numbers define

$$P(a,b) = \frac{(a+b)(a+b+1)}{2} + b, \quad I(a,b) = 4P(a,b), \quad a \to b = I(a,b),$$

and

$$R(a,b) = 4P(a,b) + 1, \qquad Q_a b = 4P(a,b) + 2, \qquad E(a,b) = 4P(a,b) + 3.$$

Truth is 0, the refutation target is 1, $NX = I(X,1)$, and $\exists_a X = N(Q_a(NX))$. In particular $I(0,0) = 0$ and $R(0,0) = 1$. Every parameter ranges over every natural number, including numbers that are not conventional syntactic formula codes.

Let $\mathcal{D}$ be the inherited dominated native calculus. Its rules are truth, exact numerical modus ponens, and theorem-only generalization:

$$\mathcal{D} \vdash 0, \qquad \frac{\mathcal{D} \vdash X \quad \mathcal{D} \vdash I(X,Y)}{\mathcal{D} \vdash Y}, \qquad \frac{\mathcal{D} \vdash X}{\mathcal{D} \vdash Q_i X}.$$

For specificity, write the seven *emitted* row functions as

$$\begin{aligned}
R_1(a,b,c) &= \exists_a E(a,b),\\
R_2(a,b,c) &= Q_a(b \to c) \to (Q_a b \to Q_a c),\\
R_3(a,b,c) &= E(a,b) \to (R(c,a) \to R(c,b)),\\
R_4(a,b,c) &= Q_a Q_b c \to Q_b Q_a c,\\
R_5(a,b,c) &= E(c,a) \to (R(c,b) \to R(a,b)),\\
R_6(a,b,c) &= (\exists_a Q_a c) \to c,\\
R_7(a,b,c) &= E(c,a) \to \big(E(c,b) \to Q_{R(a,b)} E(a,b)\big).
\end{aligned}$$

The middle argument of R_6 is unused. This explicitly respects the repository's correction that its sixth output is $\rho_6(a, c)$. The eighth family is

$$L(A, B, C, U) = ((C \to U) \to A) \to ((A \to C) \to (B \to C)), \qquad U \le A.$$

The full inherited gated seed bundle is retained literally. To specify it, put

$$\begin{aligned}
\mathcal{B}(b, k, m) &= Q_b\big(m \to NQ_b(m \to NR(b, k))\big),\\
\mathcal{S}(a, b, k) &= Q_k(R(k, a) \to R(k, b)),\\
\mathcal{C}(a, b, k, m) &= Q_b\big(R(b, k) \to NQ_a(m \to NQ_a(\mathcal{S}(a, b, k) \to \mathcal{B}(b, k, m)))\big),\\
\mathcal{J}(a, b, c) &= Q_c\big(\mathcal{C}(a, b, c, c) \to N\mathcal{B}(b, c, c)\big),\\
\mathcal{V}(a, b, c) &= R(a, b) \to Q_c R(a, b),\\
\mathrm{Bundle}(a, b, c) &= N\big(\mathcal{V}(a, b, c) \to \mathcal{J}(a, b, c)\big).
\end{aligned}$$

It is an axiom when $c \neq a$ and $c \neq b$. Define the gated output $\Gamma(a, b, c)$ to be this bundle when both inequalities hold and 0 otherwise. No support-bearing part of this bundle is deleted.

The previously established calibration of $\mathcal{D}$ is used as a premise [3, 2]: ZFC proves that a finite $\mathcal{D}$-derivation of 1 exists if and only if a finite ZFC refutation exists. This premise includes the inherited all-natural certificate recovering unrestricted L from its dominated instances. The present change does not alter any axiom or rule of $\mathcal{D}$; it changes how one command samples theorem generalization.

4.2 Exact commands and field order

At a command boundary use (t, w, d, h) for top, numerical stack, additive counter, and cache. Write $[x; w] = P(x, w)$ and use right nesting for a longer explicit list. For every natural w, let $(\mathrm{head}(w), \mathrm{tail}(w))$ be its Cantor unpairing; in particular the two projections of zero are zero.

The exact transfer command is

$$S(t, w, d, h) = \begin{cases} \mathsf{halt}, & h = 1,\\ (h, \mathrm{tail}(w), d, \mathrm{head}(w)), & h \neq 1. \end{cases}$$

It tests the *incoming cache*, not the incoming top or the newly popped head. For $v = d + x$, the exact MP command is

$$\mathsf{MP}(x)(t, w, d, h) = \begin{cases} (v, [0; w], v, 0), & I(t, v) = h,\\ (0, w, v, 0), & I(t, v) \neq h. \end{cases}$$

For $\mathsf{AX}(c; x, y)$ put

$$U = d, \qquad A = d + x, \qquad B = h + y, \qquad C = c.$$

The new exact AX equation is

$$\begin{aligned}
&t' = \Gamma(A, B, C), && d' = 0, && h' = 0,\\
&w' = [R_1(A, B, C), \ldots, R_7(A, B, C), L(A, B, C, U), Q_C h, t; w].
\end{aligned}$$

The h inside $Q_C h$ is the incoming cache, before adding y. Only this ninth head differs from the parent interface, whose ninth head was $Q_U h$. The inequality $U \le A$ holds on every natural input. The gate and first eight heads are implication codes and hence cannot equal 1. The ninth head has residue 2 modulo 4, so it is neither 0 nor 1. The saved incoming top is the tenth head.

The actual field order is

$$\mathrm{fields}(S) = [0], \qquad \mathrm{fields}(\mathsf{MP}(x)) = [1, x], \qquad \mathrm{fields}(\mathsf{AX}(c; x, y)) = [c + 2, y, x].$$

The notation after the semicolon lists the counter increment before the cache increment; the wire reads the cache increment first. These are the old YX fields, not an XY variant.

4.3 Structural macros and a strong proof compiler

Define

$$D = \mathsf{AX}(0;0,0), \qquad H = D; S^{11}, \qquad \mathsf{PUSH} = D; S^{9}; \mathsf{MP}(0).$$

Concatenation denotes execution in the displayed order. For every natural input these macros terminate, possibly by the stated halt, and satisfy

$$H(t,w,d,h) = \begin{cases} \mathsf{halt}, & t = 1, \\ (t, \mathrm{tail}(w), 0, \mathrm{head}(w)), & t \neq 1, \end{cases} \qquad \mathsf{PUSH}(t,w,d,h) = (0,[t;w],0,0).$$

To verify both equations, let $a_1, \ldots, a_8, q, t$ be the ten AX heads, where now $q = Q_0 h$. The first ten shifts test only 0, $a_1, \ldots, a_8, q$, all different from 1. After ten shifts the tuple is $(q, w, 0, t)$. The eleventh therefore has exactly the displayed top-falsity guard and pop effect. After nine shifts, instead, the tuple is $(a_8, [t;w], 0, q)$. Its final MP necessarily fails because the cache has tag two and every $I(a_8, 0)$ has tag zero. Thus PUSH never halts, even when the original top or cache is 1. In particular, H is not silently identified with the parent's cache-guarded NIP.

A strong block B_X will satisfy, for *every* pair of natural numbers t, w,

$$B_X(t,w,0,0) = \mathsf{halt} \quad \text{or} \quad (X,w,0,0). \tag{SC}$$

No theoremhood assumption is made about this arbitrary input top or stack. Define

$$\mathsf{DISCARD} = \mathsf{MP}(1); \mathsf{AX}(1;0,0); S^{10}; \mathsf{MP}(0).$$

On $(t,w,0,0)$ the first MP fails, since $I(t,1) > 0$, and returns $(0,w,1,0)$. The AX has $A = C = 1$, so its gate is zero. Ten shifts give $(Q_1 0, w, 0, 0)$. Since $Q_1 0 > 0$, the final MP fails and returns $(0,w,0,0)$. This proves (SC) for truth, with no halt. On a cleared tuple $(0,w,0,0)$ put

$$J(u) = \begin{cases} \text{empty word}, & u = 0, \\ \mathsf{MP}(u), & u > 0. \end{cases}$$

It returns $(0,w,u,0)$. The empty case is essential: $\mathsf{MP}(0)$ would succeed by $I(0,0) = 0$ and introduce an unwanted explicit zero head.

Axiom retrieval. For a retained row at position $1 \leq j \leq 7$, use

$$B_{R_j(a,b,c)} = \mathsf{DISCARD}; \mathsf{AX}(c;a,b); S^{j+1}; H^{9-j}.$$

After $j+1$ shifts the selected row is top, and exactly $9-j$ of the ten generated heads remain above the original stack. The last is the saved zero. Each H preserves the selected, non-1 implication code and removes one of these heads. The final cache is therefore zero and the original stack is restored exactly. For any desired $L(a,b,c,u)$ with $u \leq a$, use

$$B_{L(a,b,c,u)} = \mathsf{DISCARD}; J(u); \mathsf{AX}(c;a-u,b); S^{9}; H.$$

Here $U = u$, $A = a$, $B = b$, and $C = c$; the single H removes saved zero. For an enabled full bundle use

$$B_{\mathrm{Bundle}(a,b,c)} = \mathsf{DISCARD}; \mathsf{AX}(c;a,b); H^{10}.$$

Its implication-coded top is preserved as all ten generated heads are removed. These calculations apply to exact native codes, including zero-valued row instances.

The new generalization block. Given B_X, set

$$B_{Q_i X} = B_X; \mathsf{PUSH}; S; \mathsf{AX}(i;0,0); S^{10}. \tag{GEN-C}$$

If the premise block returns, PUSH gives $(0,[X;w],0,0)$ and the single shift gives $(0,w,0,X)$. That shift tests incoming cache zero, so it is valid also for $X = 1$. AX now has $C = i$ and its ninth head is exactly $Q_i X$, followed by saved zero. Ten shifts return $(Q_i X, w, 0, 0)$. This is precisely where using the private binder changes the compiler: the parent's $J(i)$ is omitted. Here $A = 0$ and $B = X$, so the AX gate need not vanish when i differs from both. It is nevertheless an implication code, and the first shift safely overwrites it. The proof does not rely on a false zero-gate assertion.

Modus ponens, with the major compiled first. Let $M = I(A, B)$, and assume strong blocks for A and M. Put

$$B_B = B_M; \mathsf{PUSH}; B_A; \mathsf{PUSH}; S; S; \mathsf{MP}(B); \mathsf{AX}(B; 0, 0); S^{11}. \tag{MP-C}$$

Unless an earlier permitted halt occurs, the first four blocks give $(0, [A, M; w], 0, 0)$. Two shifts either halt on $A = 1$ or give $(A, w, 0, M)$. The next MP succeeds by literal numerical equality and returns $(B, [0; w], B, 0)$. The cleanup AX has $A = C = B$, hence zero gate, and its saved tenth head is B. After ten shifts the tuple is $(Q_B 0, [0; w], 0, B)$. The eleventh halts if $B = 1$, or returns $(B, w, 0, 0)$ otherwise. This proves (SC). When $A = B = 0$, also $M = 0$; the successful zero-alias MP still intentionally pushes one explicit zero, and the cleanup removes exactly that zero. No nonzero-formula assumption is hidden in the argument.

Induction on a finite native derivation now gives (SC) for every theorem. A finite proof DAG may first be expanded into a finite proof tree; compiling the major before the minor introduces no circular dependency. If $\mathcal{D} \vdash 1$, the fixed finite word

$$B_1; H$$

halts from every $(t, w, 0, 0)$: either B_1 already halts, or its result has top 1, which the eleventh transfer of H detects.

4.4 Exact packing and cofinal successful candidates

For a finite field list f and residual r, let $F(f, r)$ be its right-nested Cantor pairing above r. Define the encoded proof tail by

$$\mathrm{tailcode}([]) = 0, \qquad \mathrm{tailcode}(k :: L) = 1 + F\big(\mathrm{fields}(k), \mathrm{tailcode}(L)\big).$$

At a positive proof pointer the source first decrements, then reads the command's one, two, or three fields. Thus a positive encoded tail executes its next command and leaves the indicated successor tail. A freshly fetched candidate has no outer successor: its initial command is decoded directly.

Let $r = \mathrm{tailcode}(B_1; H)$. For every natural Y choose

$$N_Y = P\big(2, P(Y, P(1, r))\big). \tag{Cofinal}$$

With the actual YX field order this first executes $\mathsf{AX}(0; 1, Y)$ and leaves residual r. The first AX terminates and resets counter and cache to zero. Its resulting top and stack may be arbitrary; the universal contract of $B_1; H$ still ensures a halt. Cantor domination gives $N_Y \geq Y$, so successful candidates lie beyond every finite startup cutoff.

Every nonhalting positive proof pointer p drains strictly: after its initial decrement, the next residual is obtained by right projections of $p - 1$, so it is at most $p - 1 < p$. Each source command terminates or halts by its all-natural macro contract. Therefore a candidate cannot block the scheduler forever. The next-candidate counter increases by one, so the scheduler eventually reaches any chosen N_Y above its initial cutoff, unless it halts earlier. This proves completeness without a finite search bound or an additional fairness assumption.

4.5 An all-natural protected-tail theorem

The actual blank initializer enters a partial AX computation, not an ordinary zero-stack root. The following schedule theorem proves that the arbitrary numerical suffix underneath its safe heads is never exposed. It depends only on the exact three field lengths and command balances, so changing the generalization binder does not change its combinatorics.

Let f be the right Cantor projection and $T_s = s(s + 1)/2$. Let $W(n)$ be the finite syntactic command word parsed from fresh candidate n, ignoring possible semantic halts, and put $\mathcal{R}(0) = []$ and $\mathcal{R}(r) = W(r - 1)$ for $r > 0$. If $\mathrm{unpair}(n) = (a, y)$, then exactly

$$W(n) = \begin{cases} S; \mathcal{R}(y), & a = 0, \\ \mathsf{MP}; \mathcal{R}(f(y)), & a = 1, \\ \mathsf{AX}; \mathcal{R}(f(f(y))), & a \geq 2. \end{cases} \tag{Schedule}$$

Payloads are still read; they are irrelevant only to the following conservative balance. Give $S, \mathsf{MP}, \mathsf{AX}$ balances $-1, 0, +10$, respectively. Successful MP actually adds a further theorem zero, which this estimate

may safely ignore. Write $b(n)$ for the total balance of $W(n)$ and $B(k) = \sum_{n<k} b(n)$. Define $g(0) = 0$ and $g(r) = b(r-1)$ for $r > 0$. Strict residual descent makes these definitions well founded. Induction also proves that $W(n)$ has at most $n+1$ commands: its residual is at most n. Consequently every prefix of $\mathcal{R}(r)$ has balance at least $-r$.

Prefix pullback. If a sequence $q(0), q(1), \ldots$ has nonnegative initial partial sums, then the sequence $q(f(0)), q(f(1)), \ldots$ does too. Indeed, increasing inputs to f enumerate the consecutive blocks $[0], [0,1], [0,1,2], \ldots$. Any initial segment consists of complete such blocks and an initial part of the next one. Apply this observation twice for $q(f(f(y)))$. We use $q(r) = g(r) + 1$, for which

$$\sum_{r<k} q(r) = 1 + B(k-1) + (k-1) \qquad (k \geq 1). \tag{Shift}$$

Thus previously established inequalities $B(j) + j \geq 0$ supply the needed nonnegative partial sums. Using unshifted g here would be invalid, since the new tag order has negative initial balances.

Every complete diagonal from $s = 2$ is prefix nonnegative. We prove this jointly with $B(k) + k \geq 0$ and $B(T_s) \geq -3$, by induction on the Cantor diagonal. The base is

$$W(0) = [S], \qquad W(1) = [\mathsf{MP}], \qquad W(2) = [S, S],$$

giving $B(0), B(1), B(2), B(3) = 0, -1, -1, -3$. For $s \geq 2$, the candidates $T_s + y$ have selector $s - y$. The first $s-1$ start with AX, the next with MP, and the last with S. The first k AX candidates have total balance

$$10k + \sum_{y<k} g(f(f(y))) \geq 9k, \qquad 0 \leq k \leq s-1,$$

by prefix pullback and (Shift). All required earlier indices are at most $s - 2 < T_s$, so this does not assume the induction conclusion. Before AX candidate y the balance is at least $9y$; its AX adds 10 and its suffix loses at most $f(f(y)) \leq y$ at any prefix. Its nonempty prefixes therefore have balance at least $8y + 10 > 0$. After the AX portion the balance is at least $9(s-1)$. The MP candidate's suffix loses at most $s-1$, leaving at least $8(s-1)$ throughout. The last S costs one and its suffix loses at most s, leaving at least

$$8(s-1) - 1 - s = 7s - 9 > 0.$$

Thus the entire diagonal is prefix nonnegative and has nonnegative final balance. Since its starting B is at least -3, all its candidate boundaries have $B(k) \geq -3$; their $k \geq 3$ then gives $B(k) + k \geq 0$. This closes the simultaneous induction for every natural diagonal and every command prefix, including recursively decoded suffixes.

Exact conservative credit at any cutoff. Given N, choose $T_s \leq N < T_{s+1}$ and set $E(N) = \max(3, T_{s+1})$. Let $d(N)$ be the negative of the minimum prefix balance, including the empty prefix, of the finite word

$$W(N); W(N+1); \cdots; W(E(N)-1).$$

Every later complete diagonal is prefix nonnegative. Hence $d(N)$ is exactly the conservative credit needed for the whole infinite syntactic schedule beginning at N, not an estimate extrapolated from a finite enumeration. With an initial positive proof pointer p, let b, m be the total and minimum-prefix balances of $\mathcal{R}(p)$. The required credit is

$$\max(0, -m, d(N) - b).$$

In particular $E(1) = 3$ and $W(1); W(2) = [\mathsf{MP}, S, S]$, so

$$d(1) = 2. \tag{Credit}$$

Protection and soundness. Represent the stack as a finite explicit list of native theorem heads above a fixed arbitrary numerical tail J. Suppose top and cache are native theorems and the explicit list has at least the required credit. Returning AX adds ten theorem heads: eight axiom instances, $Q_C h$ by theorem generalization, and the saved theorem top. Returning MP either preserves the list or adds theorem zero. Before each returning S the conservative explicit length is at least one, since its subsequent length is

nonnegative. Hence it removes an explicit theorem head and never decodes J. It transfers theorem codes; if it halts, its incoming cache 1 is already a theorem. This proves that every halt on the protected run supplies a native refutation.

The explicit length is a proof invariant, not the size of a numerical list encoding. Although $P(0,0) = 0$, the identity $\mathrm{unpair}(P(x,v)) = (x,v)$ holds for all x, v. An explicit zero wrapper may therefore be counted, pushed, and popped even when its integer representation coincides with the underlying zero tail. No injective encoding of finite list length is assumed. Defining W while ignoring halts also introduces no nonhalting assumption: when a halt occurs, the argument applies to its finite executed prefix.

Eight safe heads suffice after an arbitrary AX. There is a useful stronger bootstrap variant. Even if the incoming top and cache of the *first* AX are arbitrary, its gate is an axiom or truth, its new cache is zero, and its first eight heads are valid native axioms. The ninth head $Q_C(\text{old } h)$ need not be a theorem. Absorb it, the saved old top, and the whole old stack into the single arbitrary tail

$$J = [Q_C(\text{old } h), \text{old } t; \text{old } w].$$

There are then eight explicit safe heads above J. At $N = 1, p = 0$, (Credit) protects that suffix with a permanent conservative margin of at least six. One must not instead assume that generalizing an arbitrary dirty cache makes it a theorem.

4.6 Application to the exact measured blank frame

The separately replayed bootstrap receipt for this candidate gives the following complete correspondence. After 68,710 binary transitions from a blank tape, the machine is at dispatcher state 84, head -2, header 62, gap zero, and source address `00110010010110010`, which is primitive ordinal 58. Its logical registers, in the recorded order $(N, T, W, A, D, H, X, Y, L)$, are

$$(1, 0, 0, 0, 0, 0, 0, 18, 0),$$

with all 63 spare registers zero. The physical register named A in this tuple carries the private parameter called C in the AX equations; it is distinct from their derived formula parameter $A = d + x$. The receipt compares the entire finite set of 100 one-cells, not merely those displayed register values. Its independently replayed source trace starts at AX entry ordinal 25 with

$$(1, 0, 2, 0, 0, 0, 0, 0, 0)$$

and reaches the same source ordinal and registers after exactly 177 source primitives. These concrete execution facts are premises supplied by the bound trace and source-CFG checks, not conclusions of the present logical argument.

This synthetic entry is $\mathsf{AX}(0;0,0)$ at work tuple $(t, w, d, h) = (0, 2, 0, 0)$, with next candidate $N = 1$ and empty proof pointer. By the all-natural AX contract, its remaining computation terminates and reaches

$$t = d = h = 0, \qquad w = [R_1(0,0,0), \ldots, R_7(0,0,0), L(0,0,0,0), Q_0 0, 0; 2].$$

All ten displayed heads are native theorems. The gate is zero because $C = A = B = 0$. The ninth head is $Q_0 0$ in both parent and new interfaces, but the generalization proof above covers every other input as well. The numerical suffix 2 must remain protected: its repeated unpairing eventually yields falsity, since $2 = P(0,1)$ and $1 = P(1,0)$. The next-candidate value is still 1 and the proof pointer is still zero. The required credit is only two, so the ten safe heads leave a permanent conservative margin of at least eight. Alternatively the general eight-head argument applies with margin six without using theoremhood of the saved input fields.

Consequently the partial-AX frame suffices: the all-natural source/runtime contracts advance it to the protected root without literally evaluating its enormous later formula codes. A coincident register projection alone would not justify this step; the complete frame, source ordinal, scheduler fields, spare registers, and whole-tape trace are the separately checked physical premises.

Calibration theorem and formalization scope. Assume the exact source command equations, the all-natural binary simulation of that source, and the complete blank-to-source frame above. Then ZFC proves

$$\text{the new blank-tape machine halts} \iff \mathcal{D} \vdash 1 \iff \neg\,\mathrm{Con}(\mathrm{ZFC}).$$

The forward implication is the protected-tail theorem; the reverse is the strong compiler and cofinal candidate construction. Both are uniform finite arithmetic arguments and combine with the inherited native calibration inside ZFC. Taking negations gives the requested equivalence between nonhalting and Con(ZFC). This is an ordinary mathematical proof tied to the new literal interface and its separately checked physical certificates. It does not assert a new Lean or Rocq kernel endpoint for this exact transition table merely because an endpoint exists for an earlier 215-state table.

5 Physical verification and its exact scope

5.1 Universal primitive contracts

For each source occurrence, the checker reconstructs the entire symbolic tape frame. Every decrement occurrence has a zero and a positive case; other primitives have one case. The resulting 6,151 cases quantify over the natural register values allowed by their guards, the gap, the first spare value, and an arbitrary finite remaining spare list. They include the complete zero exterior. They are not a finite list of sample register assignments.

The proof engine uses three rule forms: literal table transitions, induction on the length of a uniform unary run, and structural induction over a finite list after checking a one-block schema on the actual table. It first rederives the four required block schemas. The fresh final run used 1,617,679 rules, covered all 430 rows, and returned a positive affine duration in every case. Each duration has thirteen slots: the constant, gap, nine primary values, the first spare value, and the remaining-list width. Every coefficient is nonnegative and the smallest constant is five.

Positive durations matter: they connect an infinite source computation to an infinite binary computation and prevent a purported simulation from using zero-time steps. Per-case correspondence also proves that a finite source halt reaches the separate binary halt.

The saved compressed records contain the ordered case metadata. The default verifier audits their identities, complete case order, affine durations, row coverage, and block schemas. With `-runtime`, it reconstructs every case from the table and compares the complete resulting records with the saved ones. These are distinct replay modes, and neither is advertised as a proof-assistant kernel check.

5.2 A complete blank-prefix witness

The finite witness starts from control zero, head zero, and an empty set of one-cells. It records and replays every one of the 68,710 literal transitions. Its endpoint is compared with a separately constructed whole frame, including the complete finite one-set and the zero exterior. A second startup reader imports none of the compiler, runtime, or primary bootstrap utilities: it executes both native and dense tables in a bounded bytearray, checks every saved delta and every source-prefix record, and compares the complete final tape. Its observed head interval is $[-12, 226]$. The source alignment is a fresh 177-primitive execution from the synthetic AX entry; it is not an identification based only on the apparent top value. The exact values and protected-tail argument are given in the calibration section above.

There is no need to execute the rest of AX numerically until its very large formula codes have all been materialized on tape. The all-natural AX termination argument and the primitive simulation advance the matched partial command to its protected root. The startup proof therefore uses a finite concrete prefix and a quantified continuation theorem, with their boundary made explicit.

5.3 Comparison with current inputs

The novelty census covers the pinned repository and all six then-current ZIP inputs under `docs/incoming`. It contains 399 complete, graph-reachable 215-state table files, 128 distinct literal tables, and 86 graph families after quotienting by arbitrary state renaming and global tape reflection. All 128 literal representatives and their provenance are included.

The comparison checker constructs the forced rooted state bijection, preserving read symbols, written symbols, and moves. A second comparison reverses every move. Every comparison with GC215 fails, with a finite conflict witness. Thus GC215 is absent from this corpus under either equivalence. This does not assert semantic inequivalence of general Turing machines, or priority over every unpublished construction.

6 Formalism-side work: an axiom restriction that survives exact codes

The new candidate itself retains the accepted numerical calculus. A separate question was whether a smaller sampler could use fewer instances of an expensive axiom. The next result succeeds without needing a translation of the set-theoretic bundle or a well-formedness test. It is not yet implemented in GC215.

6.1 A quantified axiom sampler with a strict numerical bound

Let $P(a,b) = (a+b)(a+b+1)/2 + b$, $I(a,b) = 4P(a,b)$, $F = 1$, $\neg A = I(A,F)$, and $Q_i A = 4P(i,A) + 2$. Existential quantification means the literal numerical operation $E_i A = \neg Q_i \neg A$. The sixth native axiom is

$$R_6(i,A) = (E_i Q_i A) \to A.$$

All parameters below range over natural numbers, whether or not they are images of an ordinary formula encoder.

Strict matrix restriction. Retain the native implication basis, normality R_2, modus ponens, and generalization of closed theorems. Replace the sixth axiom family by only its instances with $i < A$. The resulting numerical calculus derives every unrestricted sixth-row instance, at its original code. The weaker restriction $i \le A$ therefore has the same property.

For a proof, fix i, A, and put

$$e = i + 1, \qquad W = I(e,e), \qquad M = I(W,A).$$

The identity formula W is a theorem of K and S. Consequently the same two implication schemas prove both $A \to M$ and $M \to A$. All these are closed theorems. Generalization and R_2 transport the first implication through Q_i. Implicational contraposition, normality, and contraposition once more give monotonicity of the literal operation E_i. Thus

$$E_i Q_i A \longrightarrow E_i Q_i M.$$

The restricted axiom $R_6(i,M)$ and $M \to A$ now give $R_6(i,A)$. This argument uses neither the unrestricted sixth row nor explosion. In particular it does not silently replace relative falsity by an ordinary primitive false constant.

The restriction is met for every numerical input, including zero:

$$W = 8(i+1)(i+2), \qquad M \ge 4W = 32(i+1)(i+2) > i.$$

The last inequality has the positive polynomial witness $32i^2 + 95i + 64$. A positive padding parameter avoids the exceptional identity $I(0,0) = 0$ entirely.

The supplied independent checker verifies a forward Hilbert certificate with 107 nodes and 124 expression nodes. Its only inference and axiom kinds are K (28), S (21), MP (53), R_2 (2), closed generalization (2), and a single strictly permitted R_6 instance. It checks every exact inference and the full dependency closure. The existing dominated-R_8 certificate supplies K and S independently of R_6; the construction is therefore noncircular.

Arbitrary finite cutoffs. More generally, for any specified total primitive-recursive function g, retaining only matrices $A > g(i)$ still recovers the full sixth row. Replace e above by $g(i) + 1$. Only the padding parameter changes: all Q_i operators retain binder i, and $M \ge 32(g(i)+1)(g(i)+2) > g(i)$. For each g whose totality is formalized in ZFC, the finite proof replacement and the resulting equality of all native theorem codes are formalizable there. This claim concerns a full numerical tail for each binder, not an arbitrary sparse cofinal subset of matrices.

Source interface and remaining allocation problem. The accepted AX command sets $U = d$ and $A = d + x$. Replacing just its sixth output by $R_6(U,A)$ would enumerate all pairs with $U \le A$. The preceding theorem supplies its exact logical completeness. A desired restricted row $R_6(i,b)$ is selected from a clean source state by

$$\mathsf{DISCARD}; \mathsf{J}(i); \mathsf{AX}(0; b-i, 0); S^7; H^3.$$

All other source compiler blocks retain their contracts if the other AX outputs and reset behavior remain unchanged. An unrestricted-row proof is first translated using the 107-node certificate.

This is an actionable logical interface, not yet a smaller machine. In the current postfix emitter, U has already been consumed into the partial eighth-row formula before the sixth row is constructed. A new emission schedule or temporary-storage arrangement is still needed to turn this restriction into a physical saving. In particular, incrementing the common A input would change other axioms and is not justified by this sixth-row theorem.

The secondary reader `checks/check_quantifier_independent.py` checks the 107-node proof using independent schema matching and expands the exact Cantor polynomial. For its padding matrix M and target matrix b, it obtains

$$M - i = 128i^4 + 768i^3 + 32i^2b + 1680i^2 + 96ib + 1583i + 2b^2 + 70b + 544 > 0.$$

Every coefficient is a nonnegative integer and the constant is positive. This is a symbolic all-natural certificate, not a bounded numerical check.

The main implementation obstacle is register lifetime. In the current AX schedule, the old U has already been consumed into the eighth formula by the time the sixth row is built. A new schedule can emit $\rho_6(U, U + x)$ earlier, consume the old cache destructively, and retime the other parameter reads. A concrete source prototype reached 4,731 primitive occurrences but 217 distinct subtrees and a zero run of eight. Its shorter word is not a smaller verified machine. That source remains research, with no 215-state allocation or blank calibration claimed.

7 Other calibrated encodings and command interfaces

7.1 Sixteen calibrated formula encodings from one native calculus

There is a separate way to trade source operations without weakening the logic: reverse the physical pair order independently for each formula constructor. The ordinary stack and proof-list pairing can remain unchanged. The general all-code constructor-transport method and the separate domination argument are inherited from the pinned zero-spare-transfer report's prefix-recoding section [4]. Here they are specialized to the sixteen operand orientations and independently checked; the general method is not claimed as new.

Write $J_r(a, b) = 4P(a, b) + r$ for tags $r = 0, 1, 2, 3$, respectively implication, membership, universal quantification and equality. For an arbitrary bit vector $e \in \{0, 1\}^4$, put

$$J_r^e(a, b) = \begin{cases} 4P(a, b) + r, & e_r = 0, \\ 4P(b, a) + r, & e_r = 1. \end{cases}$$

All constructors, including constructor-valued numerical labels inside other constructors, use this same chosen interpretation. Truth and falsity remain the literal numbers zero and one.

All-natural isomorphism. Define $F_e(0) = 0$. For $n > 0$, write uniquely $n = 4P(a, b) + r$ and set

$$F_e(n) = J_r^e(F_e(a), F_e(b)).$$

Both children are strictly below n, including the cases $n = 1, 2, 3$, whose two children are zero. This is primitive-recursive course-of-values recursion. For every natural a, b and tag r,

$$F_e(J_r(a, b)) = J_r^e(F_e(a), F_e(b)).$$

The only zero-input case is $J_0(0, 0) = 0$, where the equation still holds. The map fixes one because $1 = J_1(0, 0)$. Strong induction on n now proves $F_e(F_e(n)) = n$: each tag's swap occurs twice, and both child subcodes satisfy the induction hypothesis. Thus F_e is an involution on *all* natural codes, not only conventionally well-formed formulas.

The unrestricted native calculi are exactly isomorphic under F_e. Every schema and theorem-only generalization step transports by the constructor identity; numerical MP transports by exact equality. Numerical labels transform along with formula codes. The full bundle's gate $C \neq A, B$ is preserved by injectivity, and its literal falsity one is fixed. Hence

$$\mathcal{C} \vdash n \quad \Longleftrightarrow \quad \mathcal{C}_e \vdash F_e(n).$$

The dominance bound requires a separate argument. The map does not preserve numerical order: reversing implication exchanges the codes $I(1,0) = 4$ and $I(0,1) = 8$. It would therefore be incorrect to transport $U \le A$ through F_e without further work.

Instead interpret the accepted 3,728-node dominated-row certificate using the new implication constructor. Its1,260 eighth-row leaves have fourth parameter either zero (224 leaves), the entire first parameter (164), or an implication subterm of the first parameter (872). Thus every bound uses only

$$I(0,0) = 0, \qquad I(a,b) \ge a, \qquad I(a,b) \ge b.$$

Both physical orientations satisfy these identities and inequalities. The exact MP equations and zero normalization in the certificate also remain valid. Consequently the oriented dominated calculus $\mathcal{D}_e$ recovers its unrestricted eighth row and has exactly the same theorem codes as $\mathcal{C}_e$. In particular,

$$\mathcal{D}_e \vdash 1 \iff \mathcal{C}_e \vdash 1 \iff \mathcal{C} \vdash 1 \iff \mathrm{Incon(ZFC)},$$

where the final comparison is the inherited native-calculus calibration.

This supplies sixteen safe encoding choices for source optimization. An actual edit must establish the selected constructor consistently at every formula call while preserving the stack and scheduler interfaces. An indiscriminate reversal of a shared pairing helper need not do that. Physical allocation, primitive simulation, and blank initialization remain separate obligations.

The first implementation that reverses every formula pair keeps the stack codec unchanged and uses a register-conjugated pairing macro. It costs 208 distinct source subtrees, seven more than the direct ST215 source. Its mathematical equivalence is complete; a competitive physical allocation was not obtained. This result does not justify transporting the numerical dominance inequality through a nonmonotone code permutation.

7.2 Moving the sole falsity guard to a register that is about to be overwritten

The logical proof search need not test falsity at the same command boundary as earlier candidates. This gives several command interfaces while retaining the accepted numerical calculus and its exact full AX bundle. The point is to choose a guard whose register handling can share work with the surrounding source macro.

Use the work tuple (t, w, d, h) and Cantor stack notation $[x; w] = P(x, w)$. The macro names in this subsection are local to the chosen alternative interface. Let $A_0(c; x, y)$ denote the predecessor AX command: with $U = d$, $A = d + x$, $B = h + y$, $C = c$, it returns the full gate and

$$[R_1, \dots, R_7, L(A, B, C, U), Q_U h, t; w], \qquad d = h = 0.$$

Its gate and first eight new heads have tag zero; the ninth has tag two. The enabled gate is itself an implication code, since it is the literal negation $N(V_0 \to J_0)$. In particular none of these nine heads or the gate equals native falsity 1.

A top guard on transfer gives a total proof compiler. Define

$$S_T(t, w, d, h) = \begin{cases} \mathsf{halt}, & t = 1, \\ (h, \mathrm{tail}(w), d, \mathrm{head}(w)), & t \ne 1. \end{cases}$$

Keep AX and MP unchanged. Put

$$D = \mathsf{AX}(0; 0, 0), \qquad H = D; S_T^{11}, \qquad \mathsf{PUSH} = D; S_T^{9}; \mathsf{MP}(0).$$

For every natural input these macros terminate normally and satisfy

$$H(t, w, d, h) = (t, \mathrm{tail}(w), 0, \mathrm{head}(w)), \qquad \mathsf{PUSH}(t, w, d, h) = (0, [t; w], 0, 0).$$

Indeed, the first eleven transfer guards after AX inspect only its gate, zero, its first eight heads, and its ninth head. The saved incoming top appears as output of the eleventh shift and is never tested. Thus H returns even when $t = 1$. After nine shifts PUSH has a tag-two cache; its final MP necessarily fails against a tag-zero implication.

The reset macro

$$\mathsf{DISCARD} = \mathsf{MP}(1); \mathsf{AX}(1;0,0); S_T^{10}; \mathsf{MP}(0)$$

returns $(0, w, 0, 0)$ from every $(t, w, 0, 0)$. Define $J(0)$ to be empty and $J(u) = \mathsf{MP}(u)$ for $u > 0$ on a cleared input. These give a total, exact compiler $B_X(t, w, 0, 0) = (X, w, 0, 0)$ for every native theorem X. The blocks are as follows, with $M = I(A, B)$:

$$\begin{aligned}
B_0 &= \mathsf{DISCARD}, \\
B_{R_j(a,b,c)} &= \mathsf{DISCARD}; \mathsf{AX}(c;a,b); S_T^{j+1}; H^{9-j}, && 1 \le j \le 7, \\
B_{L(a,b,c,u)} &= \mathsf{DISCARD}; J(u); \mathsf{AX}(c;a-u,b); S_T^{9}; H, && u \le a, \\
B_{\mathrm{Bundle}(a,b,c)} &= \mathsf{DISCARD}; \mathsf{AX}(c;a,b); H^{10}, \\
B_{Q_i X} &= B_X; \mathsf{PUSH}; J(i); S_T; \mathsf{AX}(i;0,0); S_T^{10}, \\
B_B &= B_M; \mathsf{PUSH}; B_A; \mathsf{PUSH}; S_T; S_T; \mathsf{MP}(B); \mathsf{AX}(B;0,0); S_T^{11}.
\end{aligned}$$

Each axiom block selects the indicated head and removes exactly the remaining generated heads through the saved zero. In GEN the lone shift tests top zero and loads X into cache immediately before AX. In MP the two shifts both test top zero and leave $(A, w, 0, M)$. The literal MP succeeds, pushing one zero. The cleanup AX has zero gate and its eleven shifts remove precisely that zero. Its last guard tests $Q_B 0$, so even $B = 1$ returns normally. The alias $I(0, 0) = 0$ is handled by this same successful MP calculation. Structural induction on the finite proof establishes the total compiler without a theoremhood condition on incoming t, w. The final word $B_1; S_T$ therefore halts.

A guard at AX entry gives the usual strong compiler. Alternatively make transfer total,

$$S_0(t, w, d, h) = (h, \mathrm{tail}(w), d, \mathrm{head}(w)),$$

and define $\mathsf{AX}^\dagger$ to halt exactly when incoming $t = 1$, otherwise performing exactly A_0. Keep MP unchanged. The displayed compiler blocks, with S_0 and $\mathsf{AX}^\dagger$ substituted, now obey the strong contract: they halt or return exactly $(X, w, 0, 0)$.

Here H and PUSH halt precisely on incoming top one. DISCARD remains total because its first MP clears the top before any guard. All axiom blocks enter AX with top zero or a selected implication code. GEN may halt in PUSH if $X = 1$, which satisfies the strong contract. MP may halt in its second PUSH if $A = 1$, or at cleanup AX if $B = 1$; otherwise every displayed state equality holds. These are all new halting cases, and the old numerical zero aliases require no additional exception. The fixed suffix $B_1; \mathsf{AX}^\dagger(0;0,0)$ halts from every top/stack pair with counter/cache zero.

The same guard may instead be placed at MP entry. With total S_0 and unguarded AX, the displayed blocks terminate normally with their exact outputs. Add a test of incoming $t = 1$ to the entry of just one chosen command. Until that test halts, execution follows exactly the total command computation; thus every block now either halts or returns its former exact output. Actual-run theoremhood makes every new halt sound. Appending that guarded command to B_1 gives completeness. In particular one may guard MP instead of AX and use the final suffix $B_1; \mathsf{MP}^\dagger(0)$. This is useful when the tested top is already being copied or cleared inside the MP implementation. It does not require inserting guards into both commands.

Soundness, protected tails, and cofinal completeness. On safe runs the tested top is always a native theorem. Both interfaces therefore halt only with a native derivation of 1. Their normal command outputs preserve theoremhood. The protected-tail balance proof also applies unchanged: the three command costs remain $-1, 0, +10$, and each returning transfer removes exactly one explicit theorem head. In the second interface an AX halt is already sound, while each returning AX still adds ten heads. Thus the existing all-natural credit theorem, including $d(1) = 2$, applies to both interfaces.

The fields remain $[0]$, $[1, x]$, and $[c + 2, y, x]$. If R is the encoded residual for the corresponding fixed refutation suffix, each candidate

$$N_Y = P\big(2, P(Y, P(1, R))\big)$$

first executes $\mathsf{AX}(0;1,Y)$ (or its guarded version), then the suffix. It either halts on that first call or resets counter/cache and executes a successful compiled refutation. Since $N_Y \ge Y$, successful candidates occur beyond every finite startup cutoff. The unchanged scheduler's positive residual strictly decreases after

decrement and field decoding, so it eventually reaches such a candidate unless it has halted earlier. Under the inherited native-calculus calibration, either source interface has halting equivalent to inconsistency of ZFC.

These are complete command-level proofs. The source implementation must still prove its exact guard and normal output contracts, scratch restoration, and wire decoding. Physical allocation, all-tape simulation, and blank initialization are separate obligations; a smaller source DAG or finite sample alone does not discharge them.

An independently chosen generalization binder. A further compatible change is to replace only AX's ninth head $Q_U h$ by $Q_C h$, keeping all eight logical rows and the saved top unchanged. Soundness still follows from unrestricted theorem generalization. H and PUSH require only the ninth head's tag two, so their contracts and all axiom and MP blocks remain valid. Writing S for the transfer operation of the chosen alternative, the GEN block simplifies to

$$B_{Q_i X} = B_X; \mathsf{PUSH}; S; \mathsf{AX}(i; 0, 0); S^{10}.$$

Before AX the state is $(0, w, 0, X)$, so the ninth head is exactly $Q_i X$ and the tenth is zero. The old counter-setting block $J(i)$ is omitted. Here the AX gate need not vanish, because $A = 0$ and $B = X$; it remains an implication code and the first transfer safely overwrites it. This detail also makes the construction valid with the top-guard transfer.

The incoming-top guard has a particularly strong compiler: every proof block returns its theorem before a final falsity-test command. A 23-primitive implementation reduced the transfer word from 35 primitives but raised the whole-source DAG to 203. In the tested allocation models it lacked two usable full decoder roles. Relocating the test to MP or AX is also logically sound, but no smaller completed table resulted here. These are alternative interfaces; the delivered GC215 still uses the original incoming-cache guard.

8 What the unsuccessful reductions establish

Construction or change	Source DAG	Status in this packet
Direct ST215 predecessor	201	Inherited complete 215-state parent
Command-tag generalization	202	GC215: complete new 215-state candidate
23-leaf top-guard transfer	203	Calibrated source; tested allocation short by two roles
Fused 51-leaf generalization prefix	206	Thirty fewer leaves; no state saving
All four formula pair orders reversed	208	Exact code calibration; larger source
Early restricted-ρ_6 sampler	217	Source prototype; no small binary candidate
Strict fourth-parameter bound $U < A$	—	I, K, Peirce proved; full basis unresolved

The source and state counts are separate throughout. Fewer primitive occurrences do not imply fewer distinct source subtrees, and fewer subtrees do not automatically free physical states whose rows also serve the runtime engine. The new GC215 construction succeeds precisely because its additional role has a tape context that permits an existing physical selector.

8.1 Strict fourth-parameter dominance is only partially recovered

The existing full dominance certificate uses $U \leq A$. Its 1,260 leaves include 164 with $U = A$, so replacing the condition by $U < A$ is not a change of notation. We found a separate finite proof of all identity, K, and Peirce instances from strict leaves and truth. Its expanded certificate has 148 proof nodes and 187 term nodes. The checker covers the zero aliases by an exhaustive symbolic zero/nonzero support argument: a nonzero proper implication context strictly dominates its fourth input, while a completely collapsed target is truth zero.

This does not recover S, composition, or the unrestricted eighth family. Bounded proof searches did not close those obligations. No machine using strict $U < A$ is claimed equiconsistent with ZFC on the strength of this fragment alone.

8.2 Why deleting the last scanner state did not work

The current physical scanner recognizes an eight-zero delimiter. A simple deletion shortens that threshold to seven, but source addresses with six zeros can temporarily acquire a seventh zero during primitive execution. The known deletion therefore fails genuine primitive contracts. The prior full census records 72 unproved contracts across 37 source occurrences: 22 CD1, 11 CD2, two CD8, and two CI1. A concrete CD1 counterexample establishes that the proposed deletion is incorrect; the other unproved contracts are not individually claimed as semantic counterexamples.

Source reassociation eliminating every run of six zeros reached 205 DAG nodes for the primary K215 source and 204 for the ST source, too expensive to offset one scanner state. We also considered the weaker demand of excluding six-zero suffixes only at increment/decrement leaves. For a full-CFG-preserving rewrite of these programs, this relaxation gives no advantage: every source leaf is graph reachable, and a non-fallthrough CB/CC/CH leaf in a left-child position would make its right sibling unreachable. Thus every control leaf ends in one, and a six-zero terminal suffix must already belong to an increment or decrement. The separate proof note states the exact structured-source assumptions. Changing those assumptions, for example by adding unreachable padding, remains a different construction problem.

8.3 Local machine synthesis has a bounded negative result

We derived sound finite-window invariants by overapproximating both choices for every tape bit entering the window. The radius-five invariant for the primary K215 table was independently checked over 87,795 configurations and 175,526 nonhalting successor obligations. Such closure is an inductive all-tape fact from the designated initial control, not an empirical bound on execution time.

Several explicitly bounded rewrite searches used these invariants: direct state folds, conditional-marker folds, two-row folds with positive local joins, and small shared selectors. The final GC215 two-sided fold search tested 19,835,232 branch candidates with a 48-step join cap and found no further state deletion. This is a negative result only for that finite candidate family. It is not a proof that 214 states are impossible.

A small positive engine result is preserved separately. The sole incoming row to state 26 is $11/0 = 0L26$, so its right neighbor is known to be zero. Consequently 26/0 may be redirected from 134 to 194: the next read is zero, and both providers have $0R17$ on zero. This exact two-step replacement removes that particular engine dependence on 134/0. It changes no state count by itself, is not part of the delivered GC215 table, and requires new allocation/runtime checks before being combined with other edits.

8.4 A rejected transfer did not receive a calibration by analogy

The rotation $R = \mathsf{NIP}; \mathsf{DROP}; \mathsf{NIP}$ admitted useful PUSH and reset macros, but the attempted proof compiler could not remove generated axiom heads below a selected theorem while preserving that theorem. Ordinary Hilbert completeness does not supply the missing stack operation. Its literal table also turned out to be a pre-existing incoming comparison artifact. It is not counted as a new or calibrated result here.

9 Reproduction, integration, and trust

The archive is self-contained for its new finite checks and runtime replay. Its inherited set-theoretic calibration is precisely pinned and attributed; it is not re-proved from the ZFC axioms in this packet. The main commands, run from the extracted directory with Python assertions enabled, are:

```
python3 verify.py
python3 verify.py -runtime
```

The first verifies the file manifest, literal construction, source/CFG identities, conditional fold, dense labels, complete blank trace, both new logical certificates, inherited dominated certificate, and corpus comparison. It audits saved runtime metadata and rederives the block schemas. The second also reconstructs all 6,151 universal primitive cases. Solver libraries are not needed. The verifier writes its results to a temporary directory unless `-work` specifies a destination.

`evidence/portable_full_replay.json` records a fresh full replay of the packaged files. `MANIFEST.json` and `SHA256SUMS` bind the final files. The paper can be rebuilt by two ordinary `pdflatex` passes in `report/`.

Its source includes the full dense table; the verifier checks all 430 printed transition entries against the delivered machine file.

The verification code is intentionally executed inherited code where marked; historical notices claiming otherwise describe earlier packets. The logical checkers validate explicit finite proof data. The symbolic runtime checker validates an induction-based proof system on exact table rows. The remaining trust boundary is ordinary mathematical review of those rules, the command arithmetic, and the pinned native-calculus calibration. A new Lean or Rocq endpoint would reduce that boundary, but is not claimed here.

Attribution

The ProveIt contributors and the earlier Wade machine/source work supply the numerical calculus, arithmetic macros, source language, physical engine, compact proof-search architecture, and earlier reductions. The dominated single-axiom certificate, direct transfer compiler, protected-credit method, and much of the symbolic verifier are inherited. The prior general constructor-transport argument is explicitly credited in the orientation section. The new work in this packet is the command-tag generalization interface, its exact source and table, the new conditional decoder fold, complete new verification, the quantifier-padding theorem and finite proof, and the scoped supplementary investigations. All inherited license and notice files are retained under `licenses/` and `reference/`.

References

[1] ProveIt contributors, *K215: 215 working states for ZFC consistency*, ordinary proof and compiled-endpoint documentation, pinned revision `34b63b3b197b0a5f285a229d91bc163fcc42ab93`. https://github.com/VladimirReshetnikov/ProveIt/tree/34b63b3b197b0a5f285a229d91bc163fcc42ab93/Computability/BusyBeaver/Research/ZFC/candidate-k215.

[2] ProveIt incoming contribution, *ST215 Direct Transfer*, `ST215_Direct_Transfer_2026-10-21.zip`, at the same pinned revision. https://github.com/VladimirReshetnikov/ProveIt/blob/34b63b3b197b0a5f285a229d91bc163fcc42ab93/docs/incoming/ST215_Direct_Transfer_2026-10-21.zip.

[3] ProveIt contributors, *Ordinary proof boundary of the dominated-rho8 machine*, K218 dominated alternative, with the inherited 3,728-node certificate and calibration references. https://github.com/VladimirReshetnikov/ProveIt/blob/34b63b3b197b0a5f285a229d91bc163fcc42ab93/Computability/BusyBeaver/Research/ZFC/candidate-k218/alternatives/dominated/proof.md.

[4] ProveIt incoming/placed contribution, *Zero-spare transfer215 and further alternatives*, prefix-recoding and raw-code calibration sections, same pin. https://github.com/VladimirReshetnikov/ProveIt/blob/34b63b3b197b0a5f285a229d91bc163fcc42ab93/Computability/BusyBeaver/Research/ZFC/candidate-k215/alternatives/zero-spare-transfer/report.tex.

A Complete dense transition table

Start in state 0 on an all-zero two-way tape, head at position 0. There are 215 working states. In each read column, wDq means write w, move in direction D, and enter q. The symbol H denotes the separate halt state 215. All listed transitions are also provided in the five-column plain-text machine file.

q	Read 0	Read 1	q	Read 0	Read 1	q	Read 0	Read 1	q	Read 0	Read 1
0	1R91	1R0	54	0R22	1R54	108	0R148	1R183	162	0L108	1R170
1	0L1	1L2	55	0L11	0R50	109	0L204	0L94	163	0R144	1R169
2	0L3	1L2	56	0R60	0L16	110	0R123	1R103	164	0R193	1R28
3	0L5	1L2	57	0R69	1R70	111	0R206	1R180	165	0R158	1R141
4	1R6	0L4	58	0R61	1R71	112	0R162	1R178	166	0R175	1R175
5	0L9	1L2	59	0R54	1R59	113	0R146	1R144	167	0R178	1R121
6	0R6	0R10	60	0R48	0R25	114	0R157	1R197	168	0R143	1R162
7	0R83	1R187	61	0L28	1L72	115	0L130	0L1	169	0R75	1R60
8	0R14	1R13	62	0L116	1L30	116	0R38	1R72	170	0R143	1R87
9	0L15	1L2	63	0R73	1R63	117	0R61	1R131	171	0R103	1R95
10	0L16	0R10	64	0R96	1R148	118	0R61	1R132	172	0R138	1R83
11	0L26	1R213	65	1R181	1L53	119	0R70	0L16	173	0R172	1R75
12	0R19	1R20	66	0R142	1R143	120	0R14	1R131	174	0R20	1R37
13	0L88	1L31	67	0R76	1RH	121	0R21	1R133	175	0R140	1L147
14	0L11	0R22	68	0L193	1R31	122	0R145	1L196	176	0R96	1R51
15	0L23	1L2	69	0R40	1R62	123	0R135	1L20	177	0R165	1R51
16	1L1	0L1	70	0L88	1R77	124	0R129	1R177	178	0R17	1L130
17	0R24	0R32	71	0R55	1L78	125	0R201	1R168	179	0R76	1R115
18	0R198	1R197	72	0R79	1R80	126	0R75	1R173	180	0R105	1R123
19	0R28	1R29	73	1R0	1R63	127	0R17	1R128	181	0L39	1L109
20	0R30	1R31	74	0R74	1R81	128	0R184	1L72	182	0R168	1R156
21	0R24	0R59	75	0L143	1L139	129	0R12	1R139	183	0R148	1R114
22	0R33	1R22	76	0R84	1R85	130	0R30	1R69	184	0R11	1R97
23	0L34	1L2	77	0R88	1L131	131	0R38	1L123	185	0R189	1R98
24	0L35	0R0	78	0L107	1R56	132	0R151	0L16	186	0R144	1R195
25	0R36	1R25	79	0R48	0R33	133	0R152	0L16	187	0R154	1R188
26	0R127	1R37	80	0R90	0L41	134	0R158	1R137	188	0R172	1R182
27	0R38	0L39	81	0R214	1R174	135	0R24	0R47	189	0R64	1R7
28	0R40	1R41	82	0R95	1R12	136	0R138	1R177	190	0R148	1R126
29	0R42	1R43	83	0R87	1R96	137	0R191	1R93	191	0R153	1R111
30	0R44	1R45	84	0R203	1R21	138	0R95	1R156	192	0R150	1R122
31	0R21	1R46	85	0R115	1R99	139	0R172	1R163	193	0R89	1R87
32	0R47	1R32	86	0R148	1R112	140	0R12	1R156	194	0R212	1R127
33	0R32	1R33	87	0R101	1R102	141	0R87	1L163	195	0R202	1R108
34	0L49	1L2	88	0R79	1R78	142	0R87	1R155	196	0R148	1R166
35	1R35	1L1	89	0R100	1R171	143	0L150	1L160	197	0R196	1R157
36	0R50	1R36	90	0R104	1R105	144	0R83	1R87	198	0R138	1R136
37	0R13	1R52	91	0R65	0R0	145	0R156	1R12	199	0R192	1R18
38	0L11	0R36	92	0R27	1R106	146	0R82	1R141	200	0R87	1R205
39	0L4	1L53	93	0R179	0L204	147	0R161	1R165	201	0R160	1R190
40	0R24	0R25	94	1L109	1L94	148	0L66	1L139	202	0R125	1R149
41	0R13	0L16	95	0R19	1R110	149	0R146	1R199	203	0R44	1R99
42	0R55	1R56	96	0R60	1R60	150	0R156	1R113	204	0L4	0L204
43	0R57	1R58	97	0R164	1R154	151	0R96	1R96	205	0R193	1R138
44	0R24	0R54	98	0R66	1R186	152	0R48	0R32	206	0R161	1R207
45	0R60	1R61	99	0R104	0L204	153	0R68	1R200	207	0R208	1R179
46	0R62	0L16	100	0R116	1R117	154	0L207	1R95	208	0R141	1R209
47	0R25	1R47	101	0R71	1R58	155	0R165	1R176	209	0R142	1R210
48	0L35	0R63	102	0R57	1R118	156	0R19	1R167	210	0R211	1R86
49	0L74	1L2	103	0R21	1R119	157	0R170	1R162	211	0R185	1R162
50	0R16	1R50	104	0R21	0L10	158	0R156	1R95	212	0R92	1R107
51	0R145	1R141	105	0R61	1R41	159	0R124	1R161	213	0R122	1R134
52	0R67	1R68	106	0R120	1R8	160	0R96	1R159	214	0R194	1R115
53	0L39	1L53	107	0R100	1R121	161	0L7	1R140			